What is a `view`?

Document #:	P2415R2
Date:	2021-09-24
Project:	Programming Language C++
Audience:	LEWG

Reply-to:	Barry Revzin <barry.revzin@gmail.com> Tim Song <t.canens.cpp@gmail.com>

Contents

§ 1 Revision History

Since [P2415R0], added wording.

§ 2 Introduction

C++20 Ranges introduced two main concepts for dealing with ranges: `range` and `view`. These notions were introduced way back in the original paper, "Ranges for the Standard Library" [N4128] (though under different names than what we have now - what we now know as `range` and `view` were originally specified as `Iterable` and `Range`[1]):

> [A Range] type is one for which we can call `begin()` and `end()` to yield an iterator/sentinel pair. (Sentinels are described below.) The [Range] concept says nothing about the type's constructibility or assignability. Range-based standard algorithms are constrained using the [Range] concept.
>
> […]
>
> The [View] concept is modeled by lightweight objects that denote a range of elements they do not own. A pair of iterators can be a model of [View], whereas a `vector` is not. [View], as opposed to

> [Range], requires copyability and assignability. Copying and assignment are required to execute in constant time; that is, the cost of these operations is not proportional to the number of elements in the Range.
>
> The [View] concept refines the [Range] concept by additionally requiring following valid expressions for an object `o` of type `O`:
>
> ```
> // Constructible:
> auto o1 = o;
> auto o2 = std::move(o);
> O o3; // default-constructed, singular
> // Assignable:
> o2 = o1;
> o2 = std::move(o1);
> // Destructible
> o.~O();
> ```
>
> The [View] concept exists to give the range adaptors consistent and predictable semantics, and memory and performance characteristics. Since adaptors allow the composition of range objects, those objects must be efficiently copyable (or at least movable). The result of adapting a [View] is a [View]. The result of adapting a container is also a [View]; the container – or any [Range] that is not already a [View] – is first converted to a [View] automatically by taking the container's `begin` and `end`.

The paper really stresses two points throughout:

— views are lightweight objects that refer to elements they do not own[2]

— views are O(1) copyable and assignable

This design got muddled a bit when views ceased to require copyability, as a result of "Move-only Views" [P1456R1]. As the title suggests, this paper relaxed the requirement that views be copyable, and got us to the set of requirements we have now in 24.4.4 [range.view]:

— views are O(1) move constructible, move assignable, and destructible

— views are either O(1) copy constructible/assignable or not copy constructible/assignable

But somehow absent from the discussion is: why do we care about views and range adaptors being cheap to copy and assign and destroy? This isn't just idle navel-gazing either, [LWG3452] points out that requiring strict O(1) destruction has implications for whether `std::generator` [P2168R3] can be a `view`. What can go wrong in a program that annotates a range as being a `view` despite not meeting these requirements?

The goal of this paper is to provide good answers to these questions.

§ 3 The need for cheap copies

N4128 asked the following question:

> ```
> auto rng = v | view::reverse;
> ```
>
> This creates a view of `v` that iterates in reverse order. Now: is `rng` copyable, and if so, how expensive is the copy operation?

Why is this question important? The initial thought to `rng` itself being cheap to copy might be that we need this requirement because we write algorithms that take views by value:

```
template <input_view V>
void some_algo(V v);
```

We could have gone that route (and we definitely do encourage people to take *specific* views by value - such as `span` and `string_view`), but that would affect the usability of range-based algorithms. You could not write `ranges::sort(v)` on a `vector<T>`, since that is not a view - you would have to write `ranges::sort(views::all(v))` or perhaps something like `ranges::sort(v.all())` or `ranges::sort(v.view())`. Either way, we very much want range-based algorithms to be able to operate on, well, ranges, so these are always written instead to take ranges by forwarding reference:

```
template <input_range R>
void some_algo(R&& r);
```

At best, we write algorithms that do require views and it's those algorithms that themselves construct the views that they need - but their API surface still takes ranges (specifically, `viewable_ranges` 24.4.5 [range.refinements]) by forwarding reference.

If we don't care about views being cheap to copy because of the desire to write algorithms that take them by value, then why do we care about views being cheap to copy?

Because we very much care about views being cheap to *construct*.

Let's go back to this example:

```
auto rng = v | views::reverse;
```

This is intended to be a lazy range adaptor - constructing `rng` here isn't intended to do any work, it's just preparing to do work in the future. It's important for this to be "cheap" - in the sense that this should absolutely not end up copying all the elements of `v`, or really doing any operation on the elements of `v`. This extends to all layering of range adaptors:

```
auto rng = v | views::some
             | views::operations
             | views::here;
```

If constructing each of these range adaptors in turn required touching all the elements of `v`, this would be a horribly expensive construct - and we haven't even done anything yet! This is why we need views to be cheap to copy - range adaptors *are* the algorithms for views, and we need to be able to pass views cheaply to those.

§ 4 Refining the view requirements

Currently, in order for a type `T` to model `view`, it needs to have O(1) move construction, move assignment, and destruction. If `T` is copyable, the copy operations also need to be O(1). What happens if a type `T` satisfies `view` (whether by it inheriting from `view_base`, inheriting from `view_interface<T>`, or simply specializing `enable_view<T>` to be `true`), yet does not actually satisfy the O(1) semantics I just laid out?

Consider:

```
struct bad_view : view_interface<bad_view> {
    std::vector<int> v;

    bad_view(std::vector<int> v) : v(std::move(v)) { }

    std::vector<int>::iterator begin() { return v.begin(); }
    std::vector<int>::iterator end()   { return v.end(); }
};

std::vector<int> get_ints();

auto rng = bad_view(get_ints()) | views::enumerate;
for (auto const& [idx, i] : rng) {
    std::print("{}. {}\n", idx, i);
}
```

`bad_view` is, as the name might suggest, a bad view. It is O(1) move constructible and move assignable, but it is not O(1) destructible. It is copyable, but not O(1) copyable (though nothing in this program tries to copy a `bad_view` - but if it did, that would be expensive!). As a result, this program is violating 16.4.5.11 [res.on.requirements]/2:

> 2 If the validity or meaning of a program depends on whether a sequence of template arguments models a concept, and the concept is satisfied but not modeled, the program is ill-formed, no diagnostic required.

Ill-formed, no diagnostic required! That is a harsh ruling for this program!

But what actually goes wrong if a program-defined `view` ends up violating the semantic requirements of a `view`? The goal of a `view` is to enable cheap construction of range adaptors. If that construction isn't as cheap as expected, then the result is just that the construction is… more expensive than expected. It would still be semantically *correct*, it's just less efficient than ideal? That's not usually the line to draw for ill-formed, no diagnostic required.

Furthermore, what actual operations do we need to be cheap? Consider this refinement:

```
struct bad_view2 : view_interface<bad_view2> {
    std::vector<int> v;

    bad_view2(std::vector<int> v) : v(std::move(v)) { }

    // movable, but not copyable
    bad_view2(bad_view2 const&) = delete;
    bad_view2(bad_view2&&) = default;
    bad_view2& operator=(bad_view2 const&) = delete;
```

```
    bad_view2& operator+(bad_view2&&) = default;

    std::vector<int>::iterator begin() { return v.begin(); }
    std::vector<int>::iterator end()   { return v.end(); }
};

std::vector<int> get_ints();

auto rng = bad_view2(get_ints())
         | views::filter([](int i){ return i > 0; })
         | views::transform([](int i){ return i * i; });
```

This whole construction involves moving a `vector<int>` twice (once into the `filter_view` and once into the `transform_view`, both moving a `vector<int>` is cheap) and destroying a `vector<int>` three times (twice when the source is empty, and once eventually when we're destroying `rng` - it's this last one that is not O(1)).

In contrast, the ordained method for writing this code is actually:

```
auto ints = get_ints(); // must stash this into a variable first
auto rng = ints
         | views::filter([](int i){ return i > 0; })
         | views::transform([](int i){ return i * i; });
```

Now, this no longer involves any moves of a `vector<int>`, since `rng` will instead be holding a `ref_view` into it, so this is in some sense cheaper. But this still, in the end, requires destroying that `vector<int>` - it's just that this cost is paid by destroying `ints` rather than destroying `rng` in this formulation. That's not meaningfully different. And moreover, there's real cost to be paid by the latter formulation: now `rng` has an internal reference into `ints`, which both means that we have to be more careful because we can dangle (not an issue in the `bad_view2` formulation) and that we have an extra indirection through a pointer which could have performance impact.

Which is ironic, given that it's the performance consideration which makes `bad_view2` bad.

Let's consider relaxing the requirements as follows:

2 `T` models `view` only if:

(2.1) — `T` has O(1) move construction; and

(2.2) — `T` has O(1) move assignment; and

(2.3) — ~~`T` has O(1) destruction~~ if `N` moves are made from an object of type `T` that contained `M` elements, then those `N` objects have `O(N+M)` destruction; and

(2.4) — `copy_constructible<T>` is `false`, or `T` has O(1) copy construction; and

(2.5) — `copyable<T>` is `false`, or `T` has O(1) copy assignment.

Or, alternatively:

(2.3) — ~~`T` has~~ an object of type `T` that has been moved from has O(1) destruction; and

In this formulation, `bad_view` is still a bad view (because it is copyable and copying it is expensive - which is important because building up a range adaptor pipeline using lvalue views will try to copy them) but `bad_view2` is actually totally fine (and indeed, it is not more expensive than the alternate formulation).

In this formulation, `std::generator<T>` is definitely a `view` that does not violate any of the semantic requirements.

This formulation has another extremely significant consequence. [N4128] stated:

> [Views] are lightweight objects that refer to elements they do not own. As a result, they can guarantee O(1) copyability and assignability.

But this would no longer *necessarily* have to be the case. Consider the following:

```
template <range R> requires is_object_v<R> && movable<R>
class owning_view : public view_interface<owning_view<R>> {
    R r_; // exposition only

public:
    owning_view() = default;
    constexpr owning_view(R&& t);

    owning_view(const owning_view&) = delete;
    owning_view(owning_view&&) = default;
    owning_view& operator=(const owning_view&) = delete;
    owning_view& operator=(owning_view&&) = default;

    constexpr R& base() & { return r_; }
    constexpr const R& base() const& { return r_; }
    constexpr R&& base() && { return std::move(r_); }
    constexpr const R&& base() const&& { return std::move(r_); }

    constexpr iterator_t<R> begin() { return ranges::begin(r_); }
    constexpr iterator_t<const R> begin() const requires range<const R>{
        return ranges::begin(r_); }

    constexpr sentinel_t<R> end() { return ranges::end(r_); }
    constexpr sentinel_t<const R> end() const requires range<const R> {
        return ranges::end(r_); }


    // + overloads for empty, size, data
};

template <class R>
owning_view(R&&) -> owning_view<R>;
```

An `owning_view<vector<int>>` would completely satisfy the semantics of `view`: it is not copyable, it is O(1) movable, and moved-from object would be O(1) destructible. All without sacrificing any of the benefit that views provide: cheap construction of range adaptor pipelines.

Adopting these semantics, along with `owning_view`, would further allow us to respecify `views::all` (24.7.5 [range.all]) as:

2 The name `views::all` denotes a range adaptor object ([range.adaptor.object]). Given a subexpression `E`, the expression `views::all(E)` is expression-equivalent to:

(2.1) — *`decay-copy`*`(E)` if the decayed type of `E` models `view`.

(2.2) — Otherwise, `ref_view{E}` if that expression is well-formed.

(2.3) — Otherwise, ~~`subrange{E}`~~ `owning_view{E}`.

The first sub-bullet effectively rejects using lvalue non-copyable views, as desired. Then the second bullet captures lvalue non-view ranges by reference and the new third bullet[3] would capture rvalue non-view ranges by ownership. This is safer and more ergonomic too.

Making the above change implies we also need to respecify `viewable_range` (in 24.4.5 [range.refinements]/5), since this concept and `views::all` need to stay in sync:

5 The `viewable_range` concept specifies the requirements of a `range` type that can be converted to a `view` safely.

```
template<class T>
  concept viewable_range =
    range<T> &&
    ((view<remove_cvref_t<T>> && constructible_from<remove_cvref_t<T>, T>)
        ||
     (!view<remove_cvref_t<T>> && ~~borrowed_range~~ (is_lvalue_reference_v<T>
        || movable<remove_reference_t<T>>)));
```

§ 5 What is a `view`?

Once upon a time, a `view` was a cheaply copyable, non-owning range. We've already somewhat lost the "cheaply copyable" requirement since views don't have to be copyable, and now this paper is suggesting that we also lose the non-owning part.

So how do you answer the question now?

There may not be a clean answer, which is admittedly unsatisfying, but it mainly boils down to:

```
auto rng = v | views::reverse;
```

If `v` is an lvalue, do you want `rng` to *copy* `v` or to *refer* to `v`? If you want it to copy `v`, because copying `v` is cheap and you want to avoid paying for indirection and potentional dangling, then `v` is a `view`. If you want to refer to `v`, because copying `v` is expensive (possibly more expensive than the algorithm you're doing), then `v` is not a view. `string_view` is a `view`, `vector<string>` is not.

§ 6 Implementation Experience

This proposal has been implemented and passes the libstdc++ testsuite (with suitable modifications).

§ 7 Proposed Wording

This also resolves [LWG3452].

Update 17.3.2 [version.syn]

```
- #define __cpp_lib_ranges                    202106L
+ #define __cpp_lib_ranges                    2021XXL
    // also in <algorithm>, <functional>, <iterator>, <memory>, <ranges>
```

Add `owning_view` to 24.2 [ranges.syn]:

```
#include <compare>              // see [compare.syn]
#include <initializer_list>     // see [initializer.list.syn]
#include <iterator>             // see [iterator.synopsis]

namespace std::ranges {
  // ...

  // [range.all], all view
  namespace views {
    inline constexpr unspecified all = unspecified;

    template<viewable_range R>
      using all_t = decltype(all(declval<R>()));
  }

  template<range R>
    requires is_object_v<R>
  class ref_view;

  template<class T>
    inline constexpr bool enable_borrowed_range<ref_view<T>> = true;

+ template<range R>
+   requires see below
+ class owning_view;
+
+ template<class T>
+   inline constexpr bool enable_borrowed_range<owning_view<T>> =
        enable_borrowed_range<T>;

  // ...
}
```

Relax the requirements on `view` in 24.4.4 [range.view]:

1 The `view` concept specifies the requirements of a `range` type that has ~~constant time move construction, move assignment, and destruction; that is, the cost of these operations is independent of the number of elements in the `view`~~ the semantic properties below, which make it suitable for use in constructing range adaptor pipelines ([range.adaptors]).

```
template<class T>
  concept view =
    range<T> && movable<T> && enable_view<T>;
```

2 `T` models `view` only if:

(2.1) — `T` has `O(1)` move construction; and

(2.2) — ~~`T` has `O(1)` move assignment~~ move assignment of an object of type `T` is no more complex than destruction followed by move construction; and

(2.3) — ~~`T` has `O(1)` destruction~~ if `N` copies and/or moves are made from an object of type `T` that contained `M` elements, then those `N` objects have `O(N+M)` destruction [*Note*: this implies that a moved-from object of type `T` has `O(1)` destruction -*end note*]; and

(2.4) — `copy_constructible<T>` is `false`, or `T` has `O(1)` copy construction; and

(2.5) — `copyable<T>` is `false`, or ~~`T` has `O(1)` copy assignment~~ copy assignment of an object of type `T` is no more complex than destruction followed by copy construction.

3 [*Example 1*: Examples of `views` are:

(3.1) — A range type that wraps a pair of iterators.

(3.2) — A range type that holds its elements by `shared_ptr` and shares ownership with all its copies.

(3.3) — A range type that generates its elements on demand.

~~Most containers are not views~~ A container such as `vector<string>` does not meet the semantic requirements of `view` since ~~destruction of~~ copying the container ~~destroys~~ copies all of the elements, which cannot be done in constant time. — *end example*]

Change the definition of `viewable_range` to line up with `views::all` (see later) in 24.4.5 [range.refinements], inserting the new exposition-only variable template *`is-initializer-list`*`<T>` [Editor's note: `remove_reference_t` rather than `remove_cvref_t` because we need to reject `const vector<int>&&` from being a `viewable_range`]:

```
template <class R>
  inline constexpr bool is-initializer-list = see below; // exposition only
```

* For a type `R`, *`is-initializer-list`*`<R>` is `true` if and only if `remove_cvref_t<R>` is a specialization of `initializer_list`.

5 The `viewable_range` concept specifies the requirements of a `range` type that can be converted to a `view` safely.

```
template<class T>
concept viewable_range =
  range<T> &&
  ((view<remove_cvref_t<T>> && constructible_from<remove_cvref_t<T>, T>) ||
   (!view<remove_cvref_t<T>> && ~~borrowed_range<T>~~
         (is_lvalue_reference_v<T> || (movable<remove_reference_t<T>> &&
        !is-initializer-list<T>))));
```

Change the last bullet in the definition of `views::all` in 24.7.5.1 [range.all.general]:

2 The name `views::all` denotes a range adaptor object ([range.adaptor.object]). Given a subexpression `E`, the expression `views::all(E)` is expression-equivalent to:

(2.1) — *`decay-copy`*`(E)` if the decayed type of `E` models `view`.

(2.2) — Otherwise, `ref_view{E}` if that expression is well-formed.

(2.3) — Otherwise, ~~`subrange{E}`~~ `owning_view{E}`.

Add a new subclause under [range.all] directly after 24.7.5.2 [range.ref.view] named "Class template `owning_view`" with stable name [range.owning.view]:

1 `owning_view` is a move-only `view` of the elements of some other `range`.

```
namespace std::ranges {
  template<range R>
    requires movable<R> && (!is-initializer-list<R>) // see
        [range.refinements]
  class owning_view : public view_interface<owning_view<R>> {
  private:
    R r_ = R();   // exposition only
  public:
    owning_view() requires default_initializable<R> = default;
    constexpr owning_view(R&& t);

    owning_view(owning_view&&) = default;
    owning_view& operator=(owning_view&&) = default;

    constexpr R& base() & noexcept { return r_; }
    constexpr const R& base() const& noexcept { return r_; }
    constexpr R&& base() && noexcept { return std::move(r_); }
    constexpr const R&& base() const&& noexcept { return std::move(r_); }

    constexpr iterator_t<R> begin() { return ranges::begin(r_); }
    constexpr sentinel_t<R> end() { return ranges::end(r_); }

    constexpr auto begin() const requires range<const R>
    { return ranges::begin(r_); }
    constexpr auto end() const requires range<const R>
    { return ranges::end(r_); }

    constexpr bool empty()
      requires requires { ranges::empty(r_); }
    { return ranges::empty(r_); }
    constexpr bool empty() const
      requires requires { ranges::empty(r_); }
    { return ranges::empty(r_); }

    constexpr auto size() requires sized_range<R>
    { return ranges::size(r_); }
    constexpr auto size() const requires sized_range<const R>
    { return ranges::size(r_); }
```

```
    constexpr auto data() requires contiguous_range<R>
    { return ranges::data(r_); }
    constexpr auto data() const requires contiguous_range<const R>
    { return ranges::data(r_); }
  };
}

constexpr owning_view(R&& t);
```

2 *Effects*: Initializes *r_* with `std::move(t)`.

§ 8 References

[LWG3452] Mathias Stearn. Are views really supposed to have strict $\mathcal{O}(1)$ destruction? https://wg21.link/lwg3452

[N4128] E. Niebler, S. Parent, A. Sutton. 2014-10-10. Ranges for the Standard Library, Revision 1. https://wg21.link/n4128

[P1456R1] Casey Carter. 2019-11-12. Move-only views. https://wg21.link/p1456r1

[P2168R3] Corentin Jabot, Lewis Baker. 2021-04-19. generator: A Synchronous Coroutine Generator Compatible With Ranges. https://wg21.link/p2168r3

[P2325R3] Barry Revzin. 2021-05-14. Views should not be required to be default constructible. https://wg21.link/p2325r3

[P2415R0] Barry Revzin, Tim Song. 2021-07-15. What is a view? https://wg21.link/p2415r0

---

1. This is why they're called *range* adaptors rather than *view* adaptors, perhaps that should change as well?↩

2. except `views::single`↩

3. the existing third bullet could only have been hit by rvalue, *borrowed*, non-view ranges. Before the adoption of [P2325R3], fixed-extent `span` was the pub quiz trivia answer to what this bullet was for. Afterwards, is there a real type that would fit here?↩